

Outlier detection & categorical encoding

Subject Big Data
Degree Software Engineering w/ AI & Data Science
Year 4

Teacher Teacher Name
Author Your Name
Date June 3, 2026

Contents

1	Outlier detection with the IQR rule	1
2	Encoding a categorical variable	2

Working dataframe used throughout:

```
1 val winesdf = spark.read.format("csv")
2   .option("header", "true").option("delimiter", "|")
3   .option("inferSchema", "true").load("/root/winequality-red-white.txt")
4
5 val double_cols = winesdf.dtypes
6   .filter { case (c, t) => t.equalsIgnoreCase("DoubleType") }.map(_._1)
```

1 Outlier detection with the IQR rule

QUESTION

Apply the following rule to identify candidate values as **outliers** for each variable. We mark as anomalous those points that have the majority of their dimensions flagged as outliers.

- Candidate outliers variable by variable using the Interquartile Range. With $IQR = Q_3 - Q_1$, a value is an outlier if it falls below $Q_1 - 1.5 \cdot IQR$ or above $Q_3 + 1.5 \cdot IQR$. Generate a new column `variable_outlier` for each variable.
- For each sample, count the dimensions in which it is anomalous and keep those with three or more.

Solution. We take the quantiles with `approxQuantile`, then fold over the numeric columns adding one `_outlier` flag column per variable.

`approxQuantile`

Computes approximate quantiles of a numeric column. For a request at probability p with relative error err , the returned value x satisfies

$$\lfloor (p - err) \cdot N \rfloor \leq \text{rank}(x) \leq \lceil (p + err) \cdot N \rceil.$$

Setting `relativeError = 0.0` computes the exact quantiles (expensive).

```
1 val out_winesdf = double_cols.foldLeft(winesdf) { case (acc, c) =>
2   val Array(q1, q3) = acc.stat.approxQuantile(c, Array(0.25, 0.75), 0.0)
3   acc.withColumn(c + "_outlier",
4     when((col(c) < (lit(q1) - lit(1.5) * (lit(q3) - lit(q1)))) ||
5       (col(c) > (lit(q3) + lit(1.5) * (lit(q3) - lit(q1)))))
6   ).otherwise(0))
7 }
```

For part (b) we sum the per-variable flags and keep the rows that are clean in all but two dimensions:

```
1 val clean_winesdf = out_winesdf
2   .withColumn("num_outliers", double_cols.map(c => col(c +
   "_outlier")).reduce(_ + _))
3   .filter(col("num_outliers") < 3)
```

2 Encoding a categorical variable

QUESTION

Encode the categorical column `style` : first build an index column `idx_style` with `StringIndexer`, then one-hot encode it.

Solution. `StringIndexer` maps the string labels to numeric indices ordered by frequency (most frequent gets index `0`); `OneHotEncoder` then turns those indices into sparse binary vectors, dropping the last category by default so the entries stay linearly independent.

```
1 import org.apache.spark.ml.feature.{StringIndexer, OneHotEncoder}
2
3 val idxr = new StringIndexer()
4   .setInputCol("style").setOutputCol("idx_style").fit(winesdf)
5 val encdr = new OneHotEncoder()
6   .setInputCol("idx_style").setOutputCol("enc_style").fit(idxr.transform(winesdf))
```